

2048 Game with User Authentication and Save/Load System

Developed by: Thinn Nu San

1. Introduction / Project Description

This project is a desktop implementation of the popular 2048 puzzle game using **Java** and **JavaFX**. The application includes a complete user management system (registration and login) and allows players to save and load their game progress. The game follows classic 2048 rules while adding persistence and user accounts.

2. Important Features

- **User Registration & Login System**
- **Persistent High Score Tracking**
- **Save & Load Game Feature**
- **Smooth 4-Direction Tile Movement** using board rotation technique
- **Visual Feedback** with color-coded tiles
- **Win (2048) and Game Over detection**
- **Clean Separation of Concerns** (MVC-like structure)

What makes this project different:

- Full user authentication instead of just anonymous play
- File-based data persistence using Java Serialization
- Efficient movement logic using board rotation (reduces code duplication)
- Professional JavaFX GUI with responsive feedback

3. System Design

Architecture:

- Main.java → Entry point
- LoginScreen.java → Authentication UI
- GameScreen.java → Core game logic and UI
- User.java → Data model
- FileManager.java → Handles all file I/O operations

Data Storage:

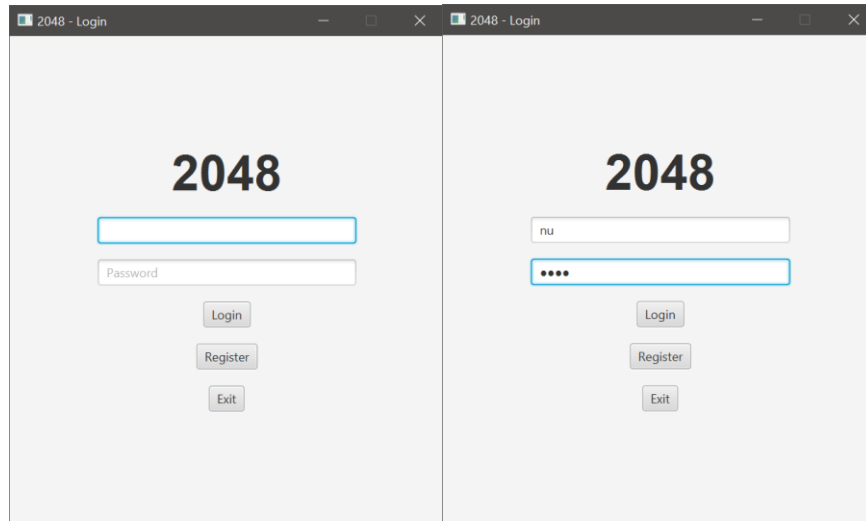
- Users and high scores → users.dat
- Game progress → savegame.dat

4. Implementation Details

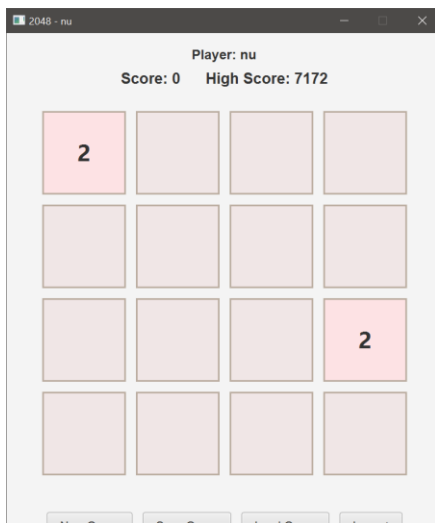
Key Technical Solutions:

- **Movement Logic:** Only moveLeft() is implemented. Other directions (Right, Up, Down) are handled by rotating the 4x4 board, moving left, then rotating back. This significantly reduces duplicate code.
- **Merging Logic:** Implemented carefully in moveLeft() to prevent double merging in one move.
- **Serialization:** User class implements Serializable to allow easy saving/loading of objects.
- **UI Design:** Used BorderPane, GridPane, and VBox for clean layout.

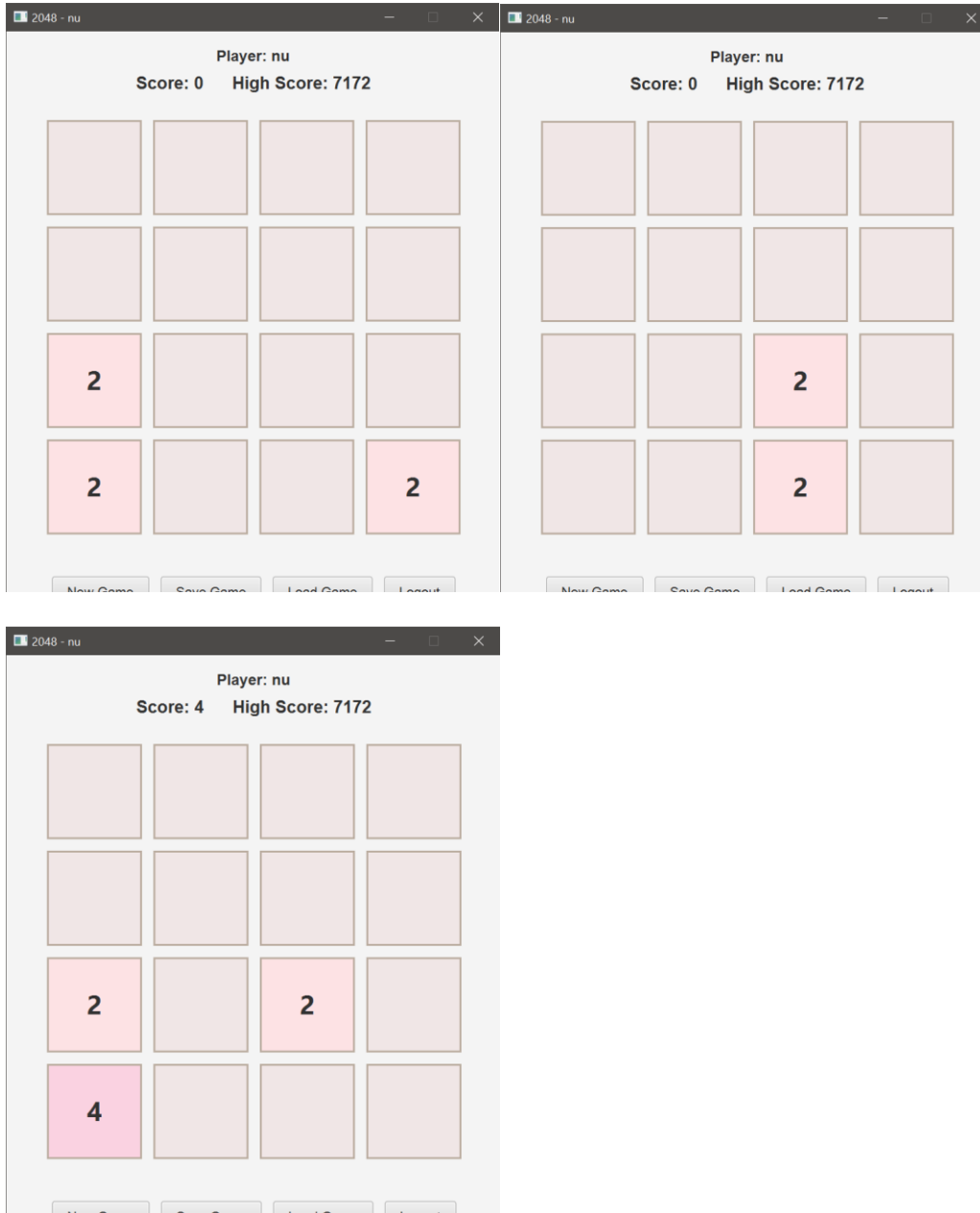
1. Login / Register Screen



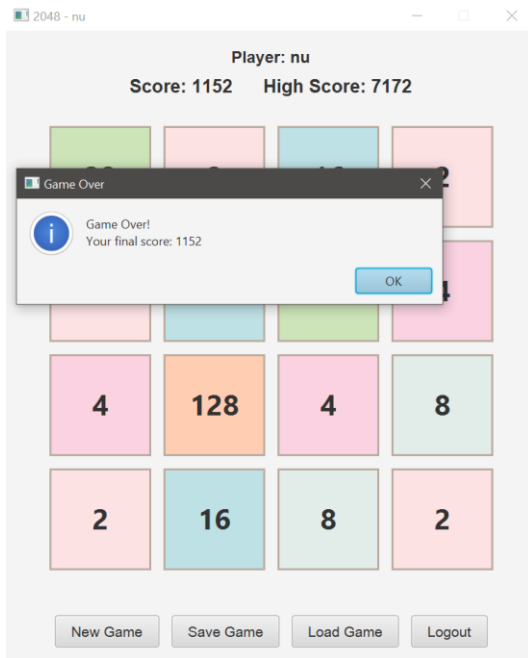
2. Main Game Screen (with score and username)



3. Successful move example



4. Game Over screen



5. Challenges & Solutions

- Implementing movement in 4 directions → Solved using board rotation technique.
- Preventing score decrease → Solved using `Math.max()` in `setHighScore()`.
- File handling errors → Handled with try-catch blocks.

6. How to Run the Project

(See GitHub README below)